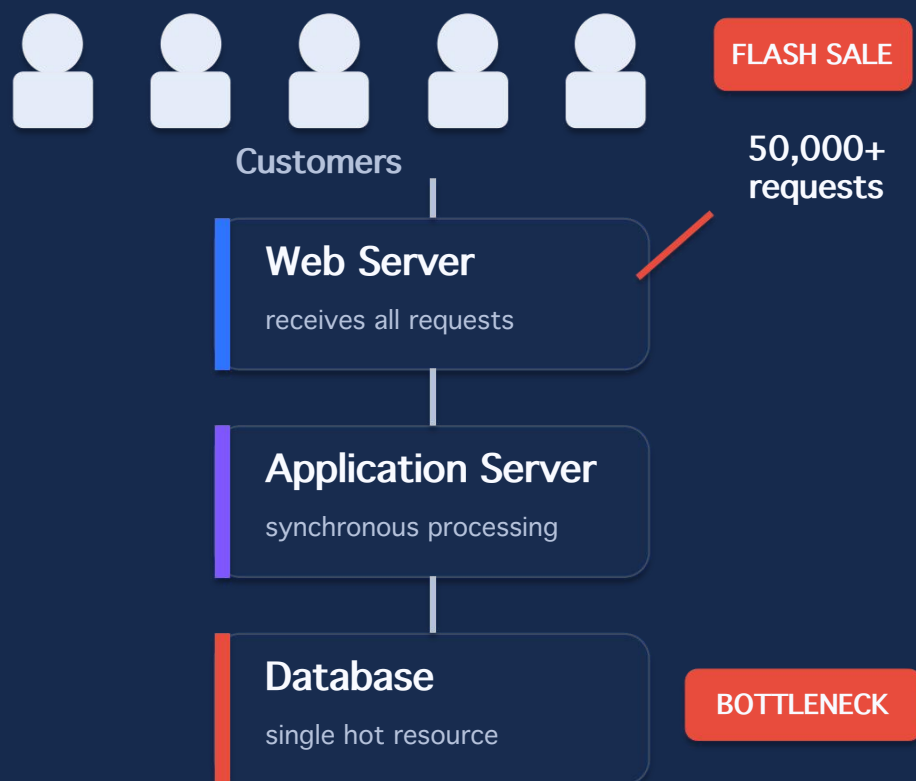


Online Ticket Seckill System — Current Architecture & Problems

Flash-sale traffic turns a simple synchronous design into a scalability and consistency problem.

CURRENT SYNCHRONOUS ARCHITECTURE



Two failure modes appear at the same time

01

Traffic spike → overload

Thousands of simultaneous reads and writes hit the same server and database, increasing latency, timeouts, and failure risk.

02

Race condition → overselling

Last ticket: A105

User A
sees AVAILABLE

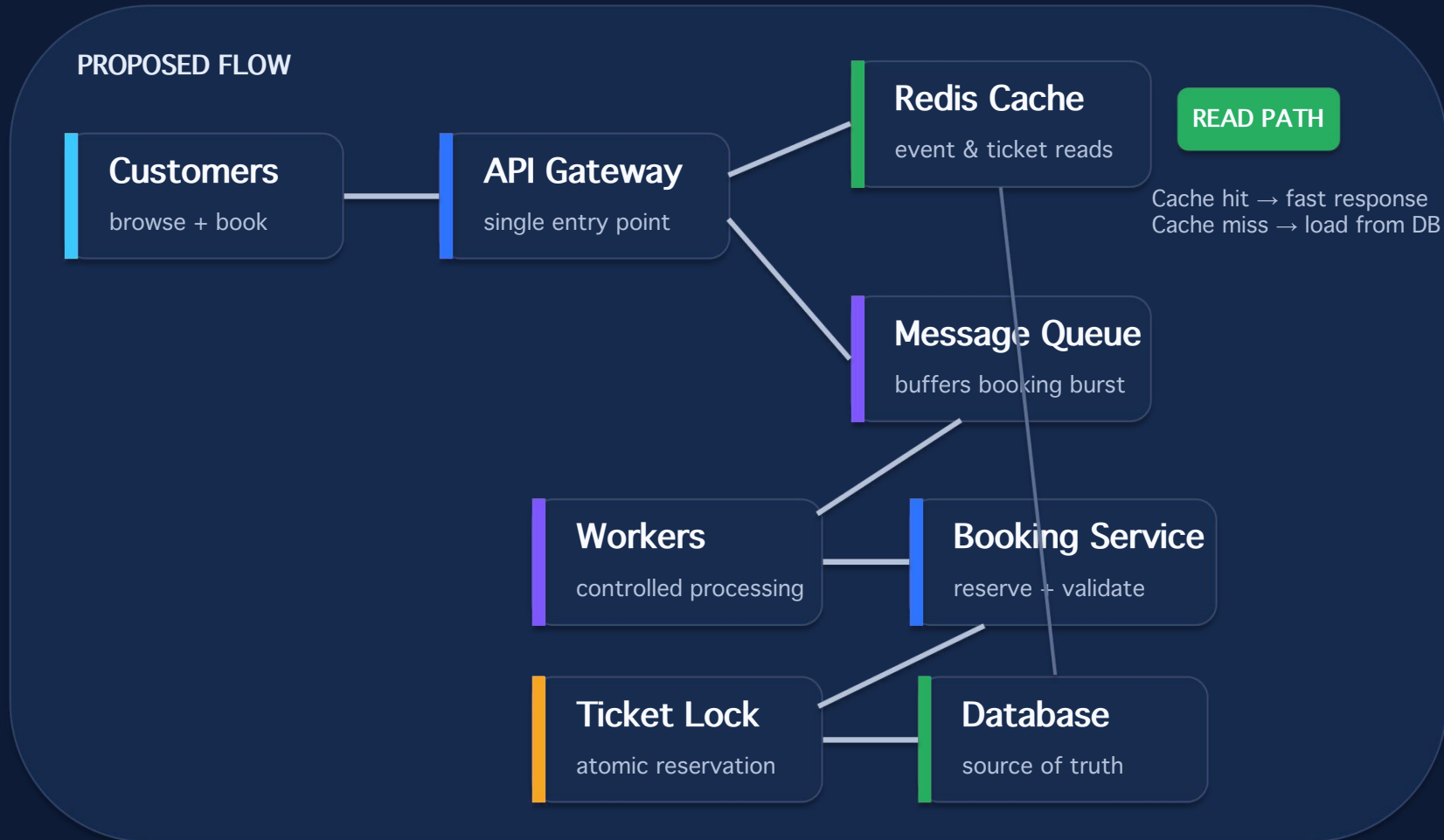
User B
sees AVAILABLE

OVERSOLD

Both transactions can read the same state before either update is committed.

Proposed Scalable Architecture

Separate read traffic, absorb bursty booking demand, and protect each ticket during the final transaction.



Why this architecture works

01

Cache-Aside

Repeated event and ticket reads are served from cache.

↓ DB READS • ↓ LATENCY

02

Queue-Based Load Leveling

Booking bursts wait in a queue and are processed at a safe rate.

ABSORBS TRAFFIC SPIKES

03

Concurrency Control

A ticket is locked / atomically reserved during purchase.

PREVENTS DOUBLE BOOKING

EXPECTED: scalable • responsive • consistent